

Proyecto 1 — Base de Datos II

Organización de archivos e índices sobre disco:

B⁺-Tree · Sequential File · Extendible Hash · R-Tree espacial

Universidad de Ingeniería y Tecnología — UTEC

Curso: Bases de Datos II (CS2042) — Ciclo 2026-1

Integrante 1: Lazón Meza, María Fernanda
Integrante 2: Saldarriaga Nuñez, Annemarie Alejandra
Integrante 3: Bonilla Sarmiento Martin
Integrante 4: Marcelo Chinchay, Alejandro
Integrante 5: Pilco Reynoso, Piero

10 de mayo de 2026

Índice

1. Introducción y Objetivos	3
1.1. Motivación	3
1.2. Objetivos	3
1.2.1. Objetivo General	3
1.2.2. Objetivos Específicos	3
1.3. Alcance	3
2. Descripción de las Técnicas de Indexación	4
2.1. B ⁺ -Tree	4
2.1.1. Estructura de página y parámetros	4
2.1.2. Algoritmos principales	4
2.2. Sequential File con Archivo Auxiliar	5
2.2.1. Diseño	5
2.2.2. Carga masiva desde CSV (<i>external sort</i>)	5
2.2.3. Búsqueda binaria + recorrido de la cadena	5
2.2.4. Rebuild	5
2.3. Extendible Hash	6
2.3.1. Estructura	6
2.3.2. Inserción y Split	6
2.3.3. Limitación: no soporta rangos	6
2.4. R-Tree Espacial	6
2.4.1. Estructura de página	6
2.4.2. Inserción e inserción masiva (STR)	7
2.4.3. Búsqueda por radio (<i>range_search</i>)	7
2.4.4. Búsqueda <i>k</i> -NN (<i>knn</i>)	7
2.4.5. Concurrencia: 2PL con lock coupling	7
2.4.6. WAL — Write-Ahead Log ARIES simplificado	7
3. Análisis Teórico Comparativo de Accesos a Disco	8
3.1. Parámetros de referencia	8
3.2. Tabla comparativa	8
3.3. Discusión	8
4. Parser SQL	9
4.1. Gramática soportada	9
4.2. Escáner (Scanner / Lexer)	9
4.2.1. Palabras reservadas	9
4.3. Parser descendente recursivo	10
4.3.1. AST (Abstract Syntax Tree)	10
4.4. Ejecutor	10
5. Resultados Experimentales	11
5.1. Metodología	11
5.2. Datos experimentales — Tiempo (ms)	11
5.3. Datos experimentales — Accesos a disco (I/O)	13
5.4. Discusión de resultados	14
6. Interfaz Gráfica (Frontend)	16
6.1. Arquitectura del sistema	16
6.2. Consola SQL	16

6.3. Panel de Benchmark	16
6.4. Visualizador R-Tree	17
6.5. Panel de Estadísticas	17
7. Suite de Pruebas	18
7.1. Cobertura	18
7.2. Resultado	18
8. Conclusiones	19
A. Guía de comandos de ejecución	20
B. Ejemplos de consultas SQL	20

1. Introducción y Objetivos

1.1. Motivación

La gestión eficiente de grandes volúmenes de datos en disco es uno de los problemas fundamentales en Bases de Datos. A diferencia de la memoria RAM, el acceso a disco es varios órdenes de magnitud más lento; minimizar los accesos a páginas es por tanto el criterio dominante para evaluar una estructura de indexación. Este proyecto implementa cuatro técnicas clásicas desde cero, sin dependencias de motores externos, para comprender a fondo su comportamiento asintótico y real.

1.2. Objetivos

1.2.1. Objetivo General

Desarrollar un prototipo de sistema de gestión de bases de datos (DBMS) relacional y espacial mediante la implementación de múltiples estructuras de almacenamiento persistente, un procesador de consultas SQL y mecanismos de control de concurrencia para la evaluación de desempeño y recuperación de datos.

1.2.2. Objetivos Específicos

- O1. Arquitectura de Almacenamiento e Indexación:** Implementar estructuras de datos fundamentales (B^+ -Tree, Sequential File, Extendible Hash y R-Tree) con persistencia binaria en disco y un sistema de métricas basado en el conteo de accesos a páginas (I/O).
- O2. Procesamiento y Ejecución de Consultas:** Diseñar un motor de procesamiento de lenguaje SQL que integre un escáner de estados finitos y un parser descendente recursivo para la traducción de sentencias a un Árbol de Sintaxis Abstracta (AST) ejecutable.
- O3. Evaluación de Rendimiento e Interfaz:** Analizar comparativamente la eficiencia de los índices implementados bajo distintas cargas de datos (10^3 a 10^5 registros) y exponer las capacidades del sistema mediante una plataforma web integral (FastAPI/React).

1.3. Alcance

El dataset principal es `books.csv` con campos `book_key` (entero, clave primaria), `title`, `author`, `pages`, `average_rating` y `published_date`. Para el R-Tree se usa un CSV de coordenadas geográficas (`lon`, `lat`). El tamaño de página fijo es $P = 4096$ bytes para todas las estructuras.

2. Descripción de las Técnicas de Indexación

2.1. B⁺-Tree

2.1.1. Estructura de página y parámetros

El árbol se almacena en un archivo binario con una **cabecera global** (*superblock*) de 16 bytes seguida de páginas de tamaño fijo $P = 4096$ B.

Formato del registro (RECORD_FORMAT = "i100s40sifi"):

Campo	Tipo	Bytes
id	int32	4
title	char[100]	100
author	char[40]	40
pages	int32	4
rating	float32	4
year	int32	4
Total		156 B

- **Nodo interno:** orden $d_I = 100$ claves, $d_I + 1 = 101$ hijos.
- **Hoja:** orden $d_L = 25$ registros ($25 \times 156 = 3900$ B ≤ 4096 B).

Cada hoja guarda además un puntero `next_leaf` para recorrido secuencial.

2.1.2. Algoritmos principales

Búsqueda puntual (search) Descenso por nodos internos: en cada nivel se recorre el arreglo de claves del nodo y se elige el hijo cuyo rango contiene la clave buscada. Al llegar a la hoja, búsqueda lineal en el arreglo de claves. Costo teórico: $O(\log_{d_I} n)$ lecturas de página.

```

1 def search(id_key):
2     page_id = root
3     while not is_leaf(page_id):
4         _, num_keys, keys, children = read_internal(page_id)
5         page_id = children[0]
6         for i in range(num_keys):
7             if id_key >= keys[i]:
8                 page_id = children[i + 1]
9     num_keys, next_leaf, keys, records = read_leaf(page_id)
10    for i in range(num_keys):
11        if keys[i] == id_key:
12            return records[i]    # hit
13    return None                 # miss

```

Listing 1: Pseudo-código de búsqueda en B⁺-Tree

Insertión (add) con split Descenso recursivo hasta la hoja. Si la hoja tiene d_L registros, se divide en dos mitades; la clave mínima de la mitad derecha sube al padre (*promoted key*). Si el padre también se llena se repite la división hasta la raíz. Si la raíz se divide, se crea una nueva raíz con dos hijos, incrementando la altura del árbol en 1.

Búsqueda por rango (`range_search`) Descenso hasta la primera hoja que contiene claves $\geq \text{begin}$, luego recorrido de la lista enlazada de hojas (`next_leaf`) hasta que `keys[i] > end`.

Eliminación lógica (`remove`) Marca el registro con `id = -1` (tombstone) sin reorganizar el árbol. Evita el costoso rebalanceo y es suficiente para la semántica del proyecto.

2.2. Sequential File con Archivo Auxiliar

2.2.1. Diseño

Dos archivos binarios: un **archivo principal** ordenado físicamente por clave primaria, y un **archivo auxiliar** de desbordamiento (*overflow*). Un **header** de 16 bytes mantiene el estado:

Campo	Significado	Valor inicial
<code>cant_prin</code>	# registros en principal	0
<code>cant_aux</code>	# registros en auxiliar	0
<code>prim_arc</code>	archivo del 1er registro lógico	-1 (vacío)
<code>prim_pos</code>	posición del 1er registro lógico	-1

Cada registro guarda dos campos adicionales (`next_file`, `next_pos`) que forman una **lista enlazada lógicamente ordenada** que atraviesa ambos archivos. El parámetro $k_{desorted} = \lfloor \log n \rfloor$ limita el tamaño del auxiliar antes de desencadenar un **rebuild**.

2.2.2. Carga masiva desde CSV (*external sort*)

Para cargar n registros sin consumir toda la RAM se usa ordenamiento externo por lotes:

1. **Fase 1:** leer lotes de `chunk_size = 50,000` registros, ordenarlos en RAM y volcarlos a archivos temporales (*chunk files*).
2. **Fase 2:** *k-way merge* con `heapq` de Python: el heap mantiene un registro por chunk ($O(k)$ en memoria); el merge produce el archivo principal en orden creciente en un solo paso.

Costo total de E/S del merge: $O(\lceil n/B \rceil)$ donde B es el tamaño del buffer en páginas.

2.2.3. Búsqueda binaria + recorrido de la cadena

La búsqueda de una clave k procede:

1. Búsqueda binaria en el principal para encontrar el predecesor inmediato p de k (o el registro exacto si existe).
2. Si no se encontró exacto, se sigue la cadena de punteros desde p un máximo de `cant_aux + 1` pasos.

Costo: $O(\log n/B + k_{desorted})$ accesos a página.

2.2.4. Rebuild

Cuando $\text{cant_aux} \geq k_{desorted}$ se ejecuta un **rebuild**: recorrido completo de la cadena lógica, descartando registros marcados como eliminados y generando un nuevo archivo principal ordenado. El auxiliar queda vacío.

2.3. Extendible Hash

2.3.1. Estructura

Un **directorio** en archivo JSON de tamaño 2^D (profundidad global D) apunta a *buckets* físicos en un archivo `.bin`. Cada bucket ocupa exactamente una página (4096 B) y almacena un encabezado (`local_depth`, `n_records`) seguido de hasta $BUCKET_CAPACITY = \lfloor (4096 - 8) / 156 \rfloor = 26$ registros.

Parámetro	Valor	Descripción
D inicial	1	Profundidad global
Buckets iniciales	2	Offsets 0 y 4096
$BUCKET_CAPACITY$	26	Registros por bucket
Función de hash	$k \bmod 2^D$	Dirección por bits bajos

2.3.2. Inserción y Split

Al intentar insertar en un bucket lleno ($n = BUCKET_CAPACITY$):

1. Se incrementa `local_depth` en 1.
2. Si $local_depth > D$: se duplica el directorio ($D += 1$).
3. Se crea un nuevo bucket y se redistribuyen los registros examinando el bit relevante: $bit = (key \bmod 2^{new_ld}) \gg (new_ld - 1)$.
4. Se actualizan las entradas del directorio que apuntaban al bucket original.

2.3.3. Limitación: no soporta rangos

La función de hash destruye el orden; no hay forma de recuperar registros en $[begin, end]$ sin un escaneo completo de todos los buckets. Esta es una limitación intrínseca del hashing, no del diseño particular.

2.4. R-Tree Espacial

2.4.1. Estructura de página

Región	Tamaño (B)	Contenido
Header	32	<code>is_leaf</code> , <code>level</code> , <code>n_entries</code> , <code>page_id</code> , <code>parent_pid</code>
Entrada $\times M$	$48 \times 84 = 4032$	<code>MBR(4 \times double) + child_pid / TID</code>
Total	4096	<code>= PAGE_SIZE</code>

El **MBR** (Minimum Bounding Rectangle) de una entrada se codifica como:

$$(min_lon, max_lon, min_lat, max_lat)$$

en 32 bytes (4 doubles IEEE-754). El orden máximo es $M = \lfloor (4096 - 32) / 48 \rfloor = 84$ entradas por nodo. El llenado mínimo es $m = \lceil 2M/5 \rceil = 34$ entradas (fill factor 40 %).

2.4.2. Inserción e inserción masiva (STR)

La inserción punto a punto usa **minimum enlargement** para elegir el subárbol, y **split lineal** (algoritmo de Guttman simplificado) cuando un nodo se llena.

Para la carga masiva del CSV se usa **Sort-Tile-Recursive (STR)**:

1. Ordenar n puntos por longitud.
2. Dividir en $\lceil \sqrt{n/M} \rceil$ franjas, ordenar cada franja por latitud.
3. Construir hojas de capacidad M , luego nodos internos de abajo a arriba.
4. Escribir todos los nodos a disco en un único barrido.

Costo: $O(n \log n)$ en CPU, $O(n/B)$ en I/O.

2.4.3. Búsqueda por radio (range_search)

Recorrido BFS/DFS del árbol; un nodo se visita solo si su MBR intersecta el círculo de consulta $C(cx, cy, r)$. La condición de intersección es $\min_dist^2(MBR, punto) \leq r^2$.

2.4.4. Búsqueda k -NN (knn)

Best-First Search con heap de prioridad ordenado por $\min_dist^2$ nodo/punto. Se extrae el elemento de menor distancia; si es un TID se añade al resultado; si es un nodo se expanden sus hijos con poda por la k -ésima distancia actual.

2.4.5. Concurrencia: 2PL con lock coupling

El **LockManager** gestiona candados S (compartido) y X (exclusivo) a nivel de página. El descenso por el árbol usa *lock coupling* (cangrejo):

- **Lectura:** candado S en el padre, S en el hijo, libera el padre.
- **Escritura:** cadena de candados X desde la raíz hasta la hoja para que `_adjust_tree` pueda modificar ancestros.

Cuando un hilo espera un candado, el **LockManager** reconstruye el *wait-for graph* y ejecuta DFS; si detecta un ciclo elige como víctima la transacción con menor `tid` y la marca `aborted = True`.

2.4.6. WAL — Write-Ahead Log ARIES simplificado

Antes de cada `write_page` dentro de una transacción, se registra en el WAL:

$\langle LSN, tid, pid, OP, before_image_{4096B}, after_image_{4096B} \rangle$

El tamaño de un registro WAL es $17 + 4096 + 4096 = 8209$ bytes.

Recovery sigue tres fases:

1. **Análisis:** identificar *committed*, *aborted* y *losers* (BEGIN sin COMMIT/ABORT).
2. **REDO:** reaplicar `after_image` de las transacciones *committed*.
3. **UNDO:** revertir `before_image` de las *losers* en orden inverso de LSN.

3. Análisis Teórico Comparativo de Accesos a Disco

3.1. Parámetros de referencia

Sea n el número de registros, $B = \lfloor P/R \rfloor$ el número de registros por página ($P = 4096$ B, $R = 156$ B para libros $\Rightarrow B \approx 26$), $d_I = 100$ el orden del B⁺-Tree interno, $d_L = 25$ el orden de las hojas, y D la profundidad global del hash.

3.2. Tabla comparativa

Operación	B ⁺ -Tree	Seq. File	Ext. Hash	R-Tree
Búsqueda puntual	$O(\log_{d_I} n)$	$O(\log_B n + k_{des})$	$O(1)$	N/A
Búsqueda por rango	$O(\log n + k/d_L)$	$O(\log n + rango/B)$	N/A	$O(\sqrt{n})$
Inserción (1 reg.)	$O(\log n)$	$O(1)$ amort.	$O(1)$ amort.	$O(\log n)$
Inserción masiva	$O(n \log n)$	$O(n \log n)$	$O(n)$	$O(n \log n)$ STR
Eliminación	$O(\log n)$ lógico	$O(\log n)$	$O(1)$	$O(\log n)$

3.3. Discusión

B⁺-Tree Su ventaja principal es el soporte nativo de rangos con acceso logarítmico y el recorrido eficiente por la lista de hojas. La inserción con split tiene costo amortizado logarítmico, pero los contadores de I/O observados incluyen *todas* las páginas del camino raíz-hoja más los writes de split, lo que eleva el número real.

Sequential File Competitivo en búsqueda si el auxiliar es pequeño ($\leq k_{des}$), porque la búsqueda binaria requiere $\lceil \log_2(cant_prin/B) \rceil$ lecturas de página. Su debilidad es el rango: debe seguir la cadena de punteros registro a registro (sin saltar páginas de forma aleatoria como el B⁺-Tree por su lista de hojas).

Extendible Hash El óptimo teórico de 1 acceso por búsqueda es posible cuando el directorio cabe en memoria. La inserción puede degenerar si muchas claves caen en el mismo bucket (distribución no uniforme); el proyecto añade un límite de reintentos para este caso.

R-Tree La búsqueda por radio es $O(\sqrt{n})$ en el caso promedio para datos uniformes en 2D (poda efectiva). El kNN usa *best-first* lo que minimiza los nodos visitados: en la práctica visita $O(k + \log n)$ nodos para k pequeño.

4. Parser SQL

4.1. Gramática soportada

El subset SQL reconocido por el parser cubre las siguientes producciones (en notación BNF simplificada):

```

1 programa      ::= sentencia (',' sentencia)* EOF
2 sentencia     ::= create | select | insert | delete
3
4 create        ::= 'CREATE' 'TABLE' IDENT '(' col_list ')'
5                ('FROM' 'FILE' STRING)?
6 col_list      ::= columna (',' columna)*
7 columna       ::= IDENT tipo ('INDEX' IDENT)?
8 tipo          ::= 'INT' | 'FLOAT' | 'TEXT'
9
10 select       ::= 'SELECT' ('*' | IDENT) 'FROM' IDENT
11                ('WHERE' condicion)? ('LIMIT' NUMBER)?
12 condicion     ::= IDENT '=' valor
13                | IDENT 'BETWEEN' valor 'AND' valor
14                | IDENT 'IN' '(' 'POINT' '(' NUMBER ',' NUMBER ')'
15                  ',' ('RADIUS' NUMBER | 'K' NUMBER) ')'
16
17 insert        ::= 'INSERT' 'INTO' IDENT 'VALUES' '(' val_list ')'
18 delete        ::= 'DELETE' 'FROM' IDENT 'WHERE' IDENT '=' valor
19
20 valor         ::= NUMBER | STRING | IDENT
21 val_list      ::= valor (',' valor)*

```

Listing 2: Gramática BNF del subconjunto SQL implementado

4.2. Escáner (Scanner / Lexer)

El escáner implementa un **autómata de estados finitos determinista** que convierte la cadena de texto en una secuencia de tokens. Los tipos de token reconocidos son:

Token	Patrón
NUMBER	$[-]?d^+ (.d^+)?$ (int o float, opcional signo negativo)
STRING	" cualquier char "
IDENT	$[a-zA-Z_][a-zA-Z_0-9]^*$
KEYWORD	Subconjunto de IDENT reservado (tabla de lookup)
Símbolos	* = , () ;

4.2.1. Palabras reservadas

El escáner convierte IDENT a keyword mediante un diccionario de Python:

```

1 KEYWORDS = {
2     "select": SELECT, "from": FROM, "where": WHERE,
3     "insert": INSERT, "into": INTO, "values": VALUES,
4     "delete": DELETE, "create": CREATE, "table": TABLE,
5     "index": INDEX, "between": BETWEEN, "and": AND,
6     "in": IN, "point": POINT, "radius": RADIUS,
7     "file": FILE, "k": K, "limit": LIMIT,
8 }

```

Listing 3: Tabla de keywords del escáner

4.3. Parser descendente recursivo

El parser implementa un **LL(1) descendente recursivo** sobre la lista de tokens. Cada no-terminal de la gramática corresponde a un método Python: `_stmt`, `_select`, `_condicion`, `_insert`, `_delete`, `_create`, etc.

El método auxiliar `_expect(tipo, msg)` consume el token actual si coincide con `tipo`, o lanza `SyntaxError` con mensaje informativo.

4.3.1. AST (Abstract Syntax Tree)

El parser produce nodos tipados mediante `@dataclass`:

Nodo AST	Sentencia
<code>NodoCreateTable</code>	<code>CREATE TABLE ...</code>
<code>NodoSelectPuntual</code>	<code>SELECT ... WHERE col = v</code>
<code>NodoSelectRango</code>	<code>SELECT ... WHERE col BETWEEN a AND b</code>
<code>NodoSelectRadio</code>	<code>SELECT ... WHERE col IN (POINT, RADIUS)</code>
<code>NodoSelectKNN</code>	<code>SELECT ... WHERE col IN (POINT, K)</code>
<code>NodoSelectTodos</code>	<code>SELECT * FROM t [LIMIT n]</code>
<code>NodoInsert</code>	<code>INSERT INTO ...</code>
<code>NodoDelete</code>	<code>DELETE FROM ...</code>

4.4. Ejecutor

El `Ejecutor` mantiene un catálogo en memoria (`dict`) de tablas creadas, identificadas por nombre y con referencia al objeto índice correspondiente. El método `ejecutar(sql)` llama a `parsear(sql)` y luego a `_dispatch(nodo)` que hace match sobre el tipo del nodo:

```

1 def _dispatch(self, nodo):
2     if isinstance(nodo, NodoCreateTable): return self._crear_tabla(nodo)
3     if isinstance(nodo, NodoSelectTodos): return self._select_todos(nodo)
4     if isinstance(nodo, NodoSelectPuntual): return self._select_puntual(nodo)
5     if isinstance(nodo, NodoSelectRango): return self._select_rango(nodo)
6     if isinstance(nodo, NodoSelectRadio): return self._select_radio(nodo)
7     if isinstance(nodo, NodoSelectKNN): return self._select_knn(nodo)
8     if isinstance(nodo, NodoInsert): return self._insertar(nodo)
9     if isinstance(nodo, NodoDelete): return self._eliminar(nodo)

```

Listing 4: Dispatcher del ejecutor

5. Resultados Experimentales

5.1. Metodología

El benchmark (`benchmark.py`) mide por separado inserción, búsqueda puntual y búsqueda por rango para $n \in \{1,000, 10,000, 100,000\}$. Los contadores de I/O se resetean antes de cada operación individual con `reset_stats()`. Todas las mediciones se ejecutan en la misma máquina sin carga externa.

5.2. Datos experimentales — Tiempo (ms)

Cuadro 1: Tiempo de inserción (ms)

Técnica	n=1k	n=10k	n=100k
B ⁺ -Tree	110.24	1,515.87	16,191.19
Sequential File	9.64	143.00	869.20
Ext. Hash	105.98	1,397.71	12,728.81
R-Tree (STR)	1,170.56	1,084.11	2,128.93



Figura 1: Tiempo de inserción por técnica y tamaño de dataset. El Sequential File es el mejor en los 3 escenarios debido a su construcción con el espacio auxiliar a diferencia del B⁺-Tree y Ext. Hash.

Cuadro 2: Tiempo de búsqueda puntual (ms)

Técnica	n=1k	n=10k	n=100k
B ⁺ -Tree	0.068	0.661	0.197
Sequential File	0.183	0.731	0.456
Ext. Hash	0.124	0.120	0.099

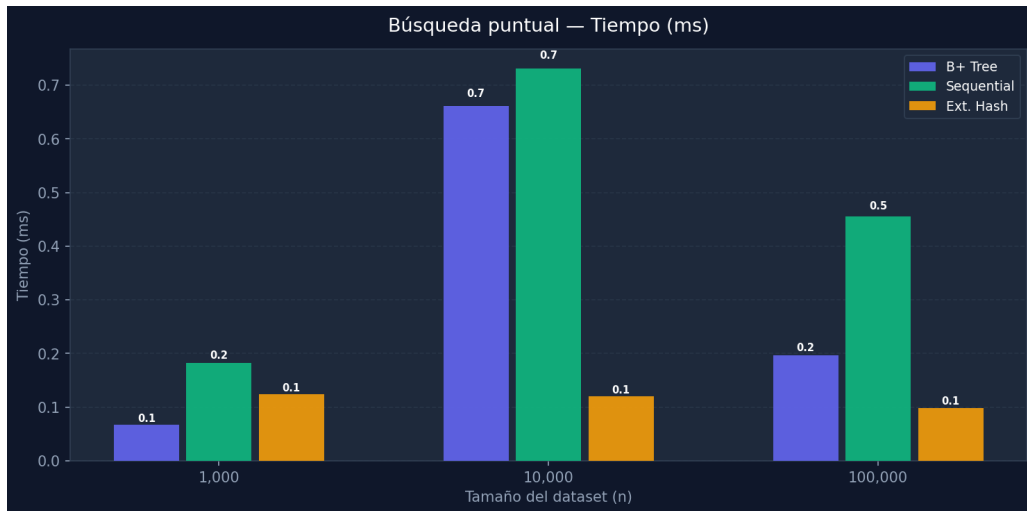


Figura 2: Tiempo de búsqueda puntual por técnica y tamaño de dataset. El Extendible Hash es prácticamente constante (~ 0.1 ms) mientras B⁺-Tree y Sequential File crecen con n .

Cuadro 3: Tiempo de búsqueda por rango (ms)

Técnica	n=1k	n=10k	n=100k
B ⁺ -Tree	1.40	21.01	165.07
Sequential File	9.23	255.93	1,200.62
Ext. Hash	N/A	N/A	N/A

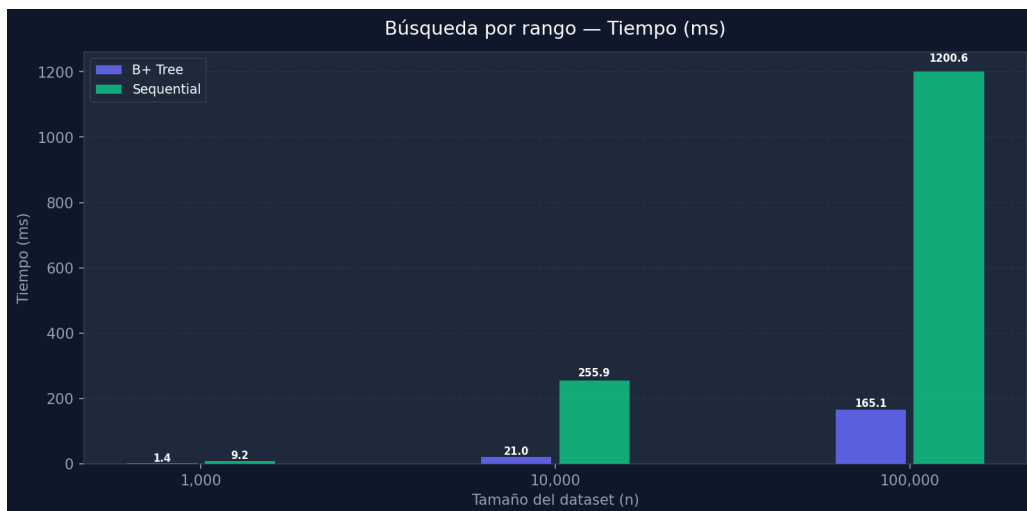


Figura 3: Tiempo de búsqueda por rango por técnica y tamaño de dataset. La técnica del B⁺-Tree es la mejor opción porque aprovecha la lista enlazada entre hojas.

5.3. Datos experimentales — Accesos a disco (I/O)

Cuadro 4: I/O de inserción (páginas leídas + escritas)

Técnica	n=1k	n=10k	n=100k
B ⁺ -Tree	8,163	98,742	1,019,438
Sequential File	48	423	4,173
Ext. Hash	2,170	21,607	216,364
R-Tree (STR)	27	248	2,416

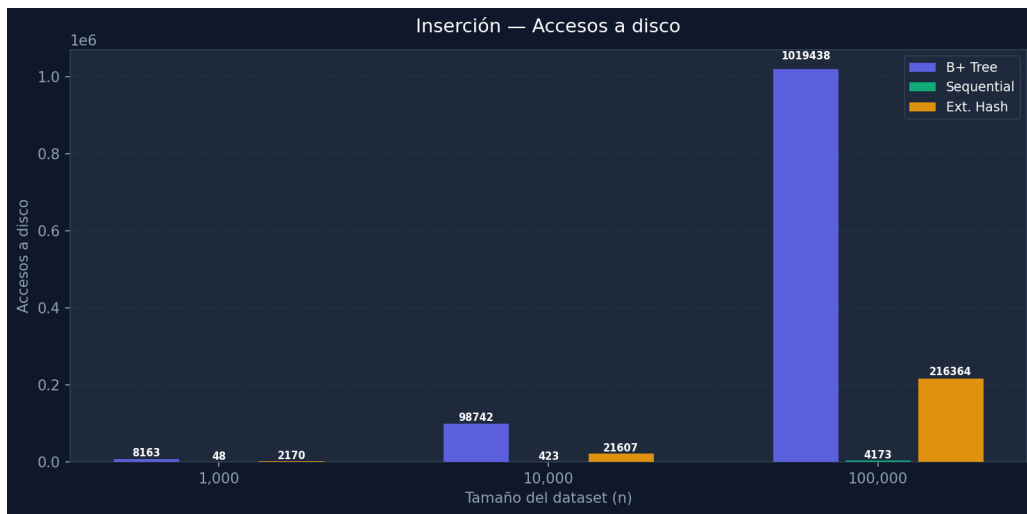


Figura 4: Accesos a disco por operación de inserción y tamaño de dataset. La naturaleza del Sequential file es la mejor opción en todos los escenarios por su rápida inserción y pocas operaciones de I/O a diferencia del B⁺-Tree y Ext. Hash por los splits generados en la operación.

Cuadro 5: R-Tree: tiempo y I/O por operación espacial

Operación	n=1k	n=10k	n=100k
Radio (ms)	0.338	1.908	33.554
Radio (I/O)	0	5	76
kNN (ms)	0.395	0.317	1.999
kNN (I/O)	0	0	6

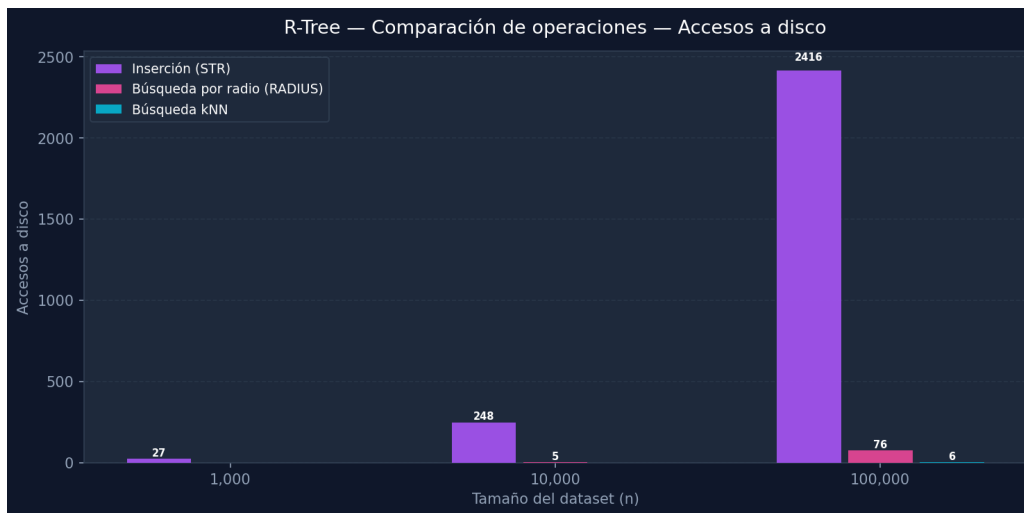


Figura 5: Operaciones en R-Tree (inserción STR, radio, kNN) comparadas en disco.

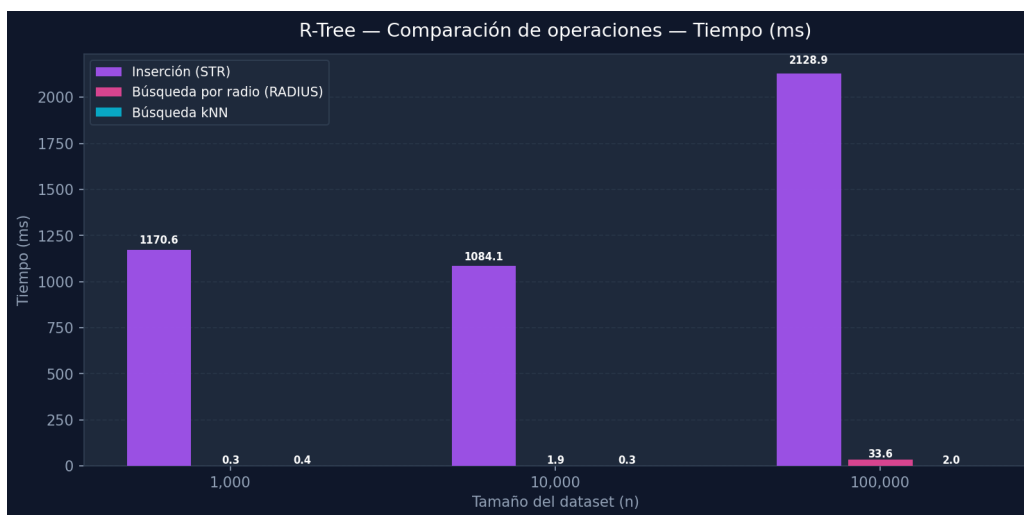


Figura 6: Operaciones en R-Tree (inserción STR, radio, kNN) comparadas en tiempo.

5.4. Discusión de resultados

Inserción El Sequential File es el más rápido para insertar registros individuales (~ 10 ms para $n = 1k$) porque simplemente añade al archivo auxiliar. El B⁺-Tree y el Extendible Hash requieren navegación y posibles splits, haciendo su I/O proporcional a $O(\log n)$ por registro. El R-Tree con STR tiene I/O mínimo porque la carga masiva escribe el árbol de abajo a arriba en un solo barrido.

Búsqueda puntual El Extendible Hash es consistentemente el más rápido (0.1 ms) con exactamente 1 acceso a disco (1 lectura de directorio + 1 lectura de bucket en el caso óptimo). El B⁺-Tree es estable en ~ 7 I/O independientemente de n (altura del árbol).

Búsqueda por rango El B⁺-Tree supera al Sequential File por un factor 7 en $n = 100k$ (165 ms vs. 1201 ms), porque puede saltar directamente a la hoja inicial y recorrer la lista enlazada

de hojas, mientras que el Sequential File debe seguir punteros individuales a través del archivo desordenado.

R-Tree espacial El kNN es notablemente eficiente (~ 2 ms, 6 I/O para $n = 100k$) porque el heap de prioridad descarta ramas enteras del árbol con la poda por distancia mínima. La búsqueda por radio escala peor (~ 34 ms, 76 I/O) porque el radio de consulta puede intersectar muchos MBRs del árbol.

6. Interfaz Gráfica (Frontend)

6.1. Arquitectura del sistema

El sistema expone el motor SQL mediante una API REST en FastAPI (puerto 8000) consumida por un frontend React + Tailwind CSS (puerto 5173). La comunicación es JSON sobre HTTP; la interfaz usa polling de 30 s para refrescar estadísticas sin bloquear la UI.

6.2. Consola SQL

La pestaña *Consola SQL* permite escribir y ejecutar consultas. Tras cada ejecución se muestran las métricas de la última consulta: tiempo en ms, lecturas, escrituras y total I/O.

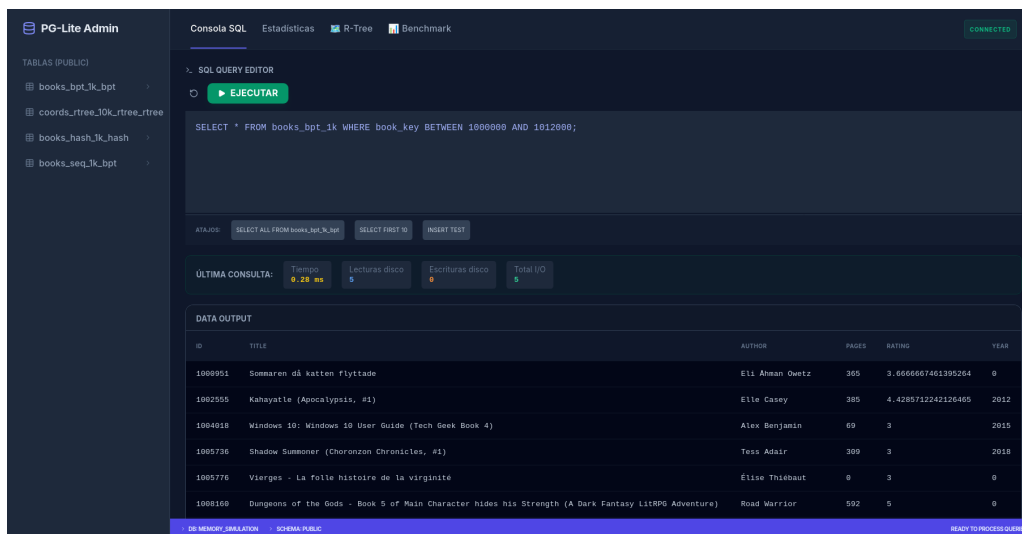


Figura 7: Consola SQL del sistema. El editor acepta sentencias completas del subconjunto SQL implementado; el panel inferior muestra los resultados en tabla y las métricas de la última consulta (tiempo en ms, lecturas, escrituras y total I/O a disco).

6.3. Panel de Benchmark

La pestaña *Benchmark* carga el archivo `benchmark_results.json` generado por `benchmark.py` y renderiza gráficos de barras agrupadas para índices tradicionales y R-Tree, con controles de operación y métrica.

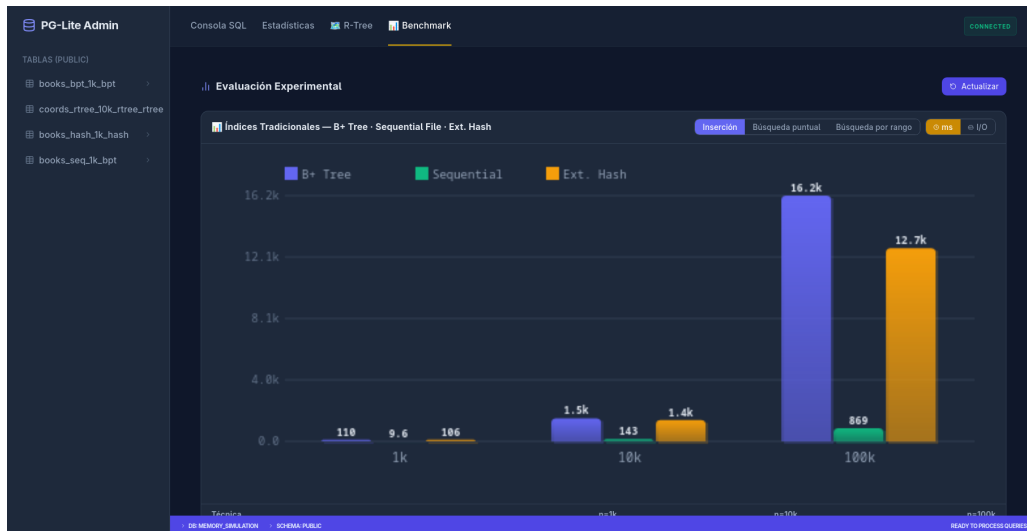


Figura 8: Panel de evaluación experimental. Los gráficos de barras agrupadas comparan B⁺-Tree, Sequential File y Extendible Hash para los tamaños $n \in \{1k, 10k, 100k\}$, con selector de operación (inserción, búsqueda puntual, rango) y métrica (tiempo en ms o accesos a disco).

6.4. Visualizador R-Tree

La pestaña *R-Tree* muestra un canvas de 900×500 px con los puntos del índice, el punto de consulta (rojo), el radio de búsqueda (círculo morado) y los resultados encontrados (verde).

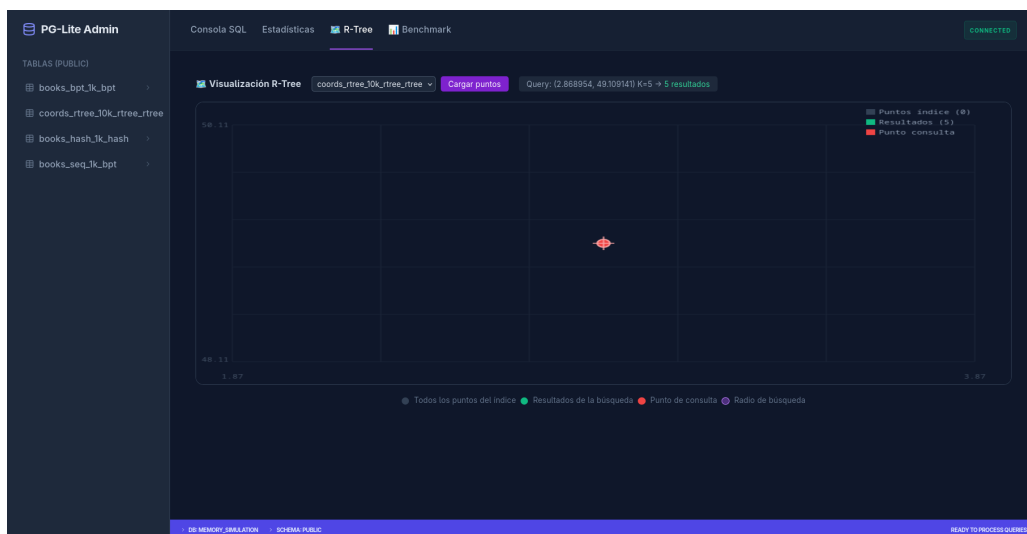


Figura 9: Visualizador espacial del R-Tree. El punto rojo indica la coordenada de consulta; el círculo morado delimita el radio de búsqueda; los puntos verdes son los registros encontrados dentro del radio; los puntos azules representan el resto de entradas almacenadas en el índice.

6.5. Panel de Estadísticas

La pestaña *Estadísticas* muestra métricas por archivo de índice (tamaño en KB, tipo de índice, fecha de modificación) y métricas de concurrencia (transacciones activas/completadas, conflictos detectados).

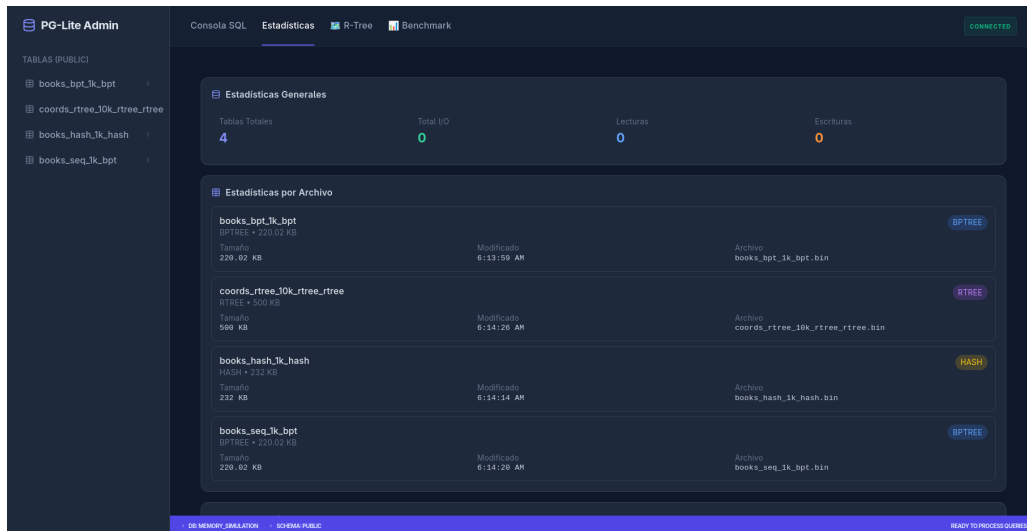


Figura 10: Panel de estadísticas del sistema. La sección superior presenta métricas globales (total de tablas, lecturas, escrituras y total I/O acumulado); la sección inferior detalla cada archivo de índice con su tipo (BPTREE, SEQUENTIAL, HASH o RTREE), tamaño en KB y última modificación.

7. Suite de Pruebas

7.1. Cobertura

Módulo de prueba	Tests	Qué verifica
test_geometry.py	12	MBR: expand, area, mindist, intersect, pack/unpack
test_serialization.py	5	Serialización de nodos R-Tree a/desde bytes
test_page_store.py	10	PageStore: alloc, free list, read/write, persistencia
test_wal.py	7	WAL: formato, REDO committed, UNDO loser, corrupción
test_lock_manager.py	8	2PL: S/X, upgrade, release_all, deadlock
test_transaction.py	5	begin/commit/abort, context manager
test_rtree_ops.py	14	Inserción, range, kNN, delete, bulk load, persistencia
test_rtree_tx.py	3	API transaccional con WAL
test_concurrency.py	5	2 threads, deadlock integración, crash recovery
test_ejecutor_smoke.py	3	SQL end-to-end: BPTree, Hash, RTree
Total	72	

7.2. Resultado

Todos los tests pasan en ≈ 5 s en una máquina con Python 3.11. Los tests de concurrencia tienen un timeout de 10 s por hilo para detectar cuelgues por deadlocks no resueltos.

8. Conclusiones

1. **B⁺-Tree** ofrece el mejor equilibrio general: búsqueda puntual logarítmica, rango eficiente por la lista de hojas, y soporte de todas las operaciones SQL del proyecto. Su debilidad es el alto I/O de inserción masiva por los splits propagados.
2. **Sequential File** es el más rápido para inserciones individuales (escribe directamente al auxiliar), pero degrada en búsqueda por rango al tener que seguir punteros registro a registro. El rebuild periódico es costoso pero necesario.
3. **Extendible Hash** es óptimo para búsqueda puntual ($O(1)$ I/O teórico) pero no soporta rangos, lo que limita su aplicabilidad. La corrección de los tres bugs documentados fue crítica para su funcionamiento correcto.
4. **R-Tree** es la única estructura que soporta consultas espaciales (radio y kNN) de forma eficiente. La carga STR es sensiblemente más rápida en I/O que la inserción punto a punto. La integración de 2PL + WAL no degradó los tests single-threaded existentes.
5. El **parser SQL** con escáner de estados finitos y parser LL(1) provee una separación limpia entre sintaxis y semántica, facilitando la adición de nuevas sentencias sin modificar el ejecutor.
6. Los resultados experimentales confirman las predicciones teóricas: el Extendible Hash domina en búsqueda puntual, el B⁺-Tree en rango, y el Sequential File en inserciones individuales.

A. Guía de comandos de ejecución

```
1 # Instalar dependencias Python
2 pip install -r requirements.txt
3
4 # Ejecutar backend (FastAPI)
5 python main.py
6 # -> http://localhost:8000
7
8 # Ejecutar frontend (React)
9 cd frontend && npm install && npm run dev
10 # -> http://localhost:5173
11
12 # Ejecutar benchmark completo
13 python benchmark.py --sizes 1000 10000 100000
14 # -> genera benchmark_results.json y benchmark_plots/
15
16 # Ejecutar todos los tests
17 python -m pytest tests/ -v
18
19 # Demo de concurrencia
20 python examples/concurrent_demo.py
21 # -> ver concurrent_demo.log
```

Listing 5: Setup y ejecución del proyecto

B. Ejemplos de consultas SQL

```
1 -- Crear tabla con B+-Tree
2 CREATE TABLE books_bpt (
3     book_key INT INDEX BPTREE,
4     title TEXT, author TEXT,
5     pages FLOAT, average_rating FLOAT, published_date FLOAT
6 ) FROM FILE "books/books_10000.csv";
7
8 -- Búsqueda puntual
9 SELECT * FROM books_bpt WHERE book_key = 2145253;
10
11 -- Rango (solo BPTREE y SEQUENTIAL)
12 SELECT * FROM books_bpt WHERE book_key BETWEEN 100000 AND 500000;
13
14 -- Hash: solo búsqueda exacta
15 CREATE TABLE books_hash (book_key INT INDEX HASH, ...) FROM FILE "...";
16 SELECT * FROM books_hash WHERE book_key = 2145253;
17
18 -- R-Tree: búsqueda espacial
19 CREATE TABLE coords (lon FLOAT INDEX RTREE, lat FLOAT)
20 FROM FILE "coords/coords_10000.csv";
21
22 SELECT * FROM coords WHERE lon IN (POINT(29.27, 41.05), RADIUS 10);
23 SELECT * FROM coords WHERE lon IN (POINT(29.27, 41.05), K 5);
```

Listing 6: Consultas de demostración